
TECHNICAL DOCUMENTATION PACK · DOCUMENT 02

LAAM

System Architecture

Components, request flows, the agent and workflow engines, the model and connector layers, the data model and the integration surface of a self-hosted, multi-user AI work assistant.



VERSION
1.0

DATE
2026-09-24

CODE BASELINE
LAAM @ 7eb0724

Contents

1 Overview and design principles

2 System context

3 Component architecture

4 Chat and agent engine

Turn lifecycle · prompt and persona precedence · tool loop · streaming protocol · reasoning effort · context management

5 Model provider layer

6 Tools and connectors

Internal tools · built-in connectors · MCP client · result handling

7 Reliability guards

8 Workflow engine

Graph model · execution modes · durability and idempotent writes · triggers and scheduler

9 AI workflow authoring

10 Voice pipeline

11 Data model

12 API surface

13 Evaluation harness

A Appendix – API endpoint reference

Status vocabulary used throughout: **LIVE** implemented, wired and exercised with real data · **BUILT** implemented and wired, not yet exercised with production data · **PLANNED** roadmap, flag-off by default, or interface only. Deployment status: **pre-production** – LAAM runs today as a self-hosted Docker Compose stack for development, internal use and demonstrations.

1 Overview and design principles

LAAM is a self-hosted, multi-user AI work assistant. People ask business questions in chat or by voice; LAAM answers them against live data by calling tools on connected systems. In chat, every write to a connected system waits for the user to confirm a code-built preview; scheduled workflows may write only through tools marked workflow-safe and only to allow-listed recipients. The same tool layer powers durable, schedulable workflow graphs that an AI can author and test-run on the user's behalf.

LAAM is a single Next.js 16 application (React 19, TypeScript strict) backed by PostgreSQL 16. It ships seven product surfaces:

Surface	What it does	Status
Chat /chat	Multi-provider LLM assistant with tool calling over connectors and MCP servers, write-confirm cards, file attachments (PDF, DOCX, images, OCR), rich rendering (tables, charts, maps) and per-user workspaces with a shared persona.	LIVE
Voice /constellation	Full-screen, hands-free voice command centre: browser speech recognition, voice-activity barge-in, streaming neural text-to-speech from a self-hosted service.	LIVE
Workflows /workflows	Durable graph automations (agent, connector, MCP, condition, for-each nodes; optional parallel execution), cron schedules, dry-run, and AI authoring that test-runs the graphs it proposes. The workflow engine is LIVE; the cron scheduler and AI authoring are BUILT.	LIVE / BUILT
Connectors /connectors	12 built-in connectors (11 integrations and 1 offline demo) plus any number of per-user remote MCP servers, with encrypted per-user credentials and OAuth where the provider supports it.	LIVE / BUILT
Search /search	Global search across monitored sessions, the caller's conversations (including message content) and workflows, using trigram indexes.	BUILT
Access /settings/*	Role-based access control (owner / admin / member / viewer), off-boarding, personal API and MCP access tokens, admin-provisioned keys, per-user custom agents.	LIVE
Monitoring /monitoring	Real-time view of AI coding-agent sessions (ingested from transcript files by a collector) and of LAAM's own chat and workflow runs, including model, tools, tokens and cost.	LIVE

LAAM is also an MCP server **BUILT**: external agents holding an access token can call nine read-only tools over `POST /api/mcp`.

26 PAGE ROUTES	88 API METHOD HANDLERS (64 ROUTE FILES)	22 POSTGRES TABLES · 19 MIGRATIONS	363 TEST FILES (VITEST)
12 BUILT-IN CONNECTORS · 46 TOOLS	12 INTERNAL AGENT TOOLS	5 LLM PROVIDERS SUPPORTED	3 UI LANGUAGES (VI · EN · ZH)

All figures counted from the source tree at commit `7eb0724` . The 12 built-in connectors are 11 integrations and 1 offline demo; 14 of their 46 tools are writes. The Vitest suite holds 3,573 test cases in 363 files (3,567 passing when re-run on the baseline, 2026-09-24).

■ **Design principles**

Five principles recur through the code and explain most of the architecture described in later chapters. Each is backed by code or tests rather than by convention alone.

Principle	How it is enforced
Use the model only for judgement	Routing, retries, extraction, arithmetic, table building and write execution are done by code. The model decides <i>which</i> tool to call and how to phrase the answer. Example: result tables are built from the tool's rows by code and streamed as a separate frame, so the model never retypes numbers.
Human confirms writes in chat	A tool counts as a read only when it is declared read-only by a built-in connector, or by an MCP server the user has marked trusted; everything else is treated as a write. In chat, every write to a connected system suspends the turn and waits for the user to confirm a code-built preview; the confirmed call runs in code with the sealed arguments, and its nonce is checked against the audit log. Scheduled and unattended workflows may execute writes only for tools marked workflow-safe and only to allow-listed recipients; all other writes fail closed.
Fail loud	No silent provider fallback, no blank replies, no "success" without an executed write. Each failure path produces a coded, tri-lingual notice or a failed step that names the cause.
Model-provider-agnostic	Five providers sit behind one internal adapter contract. Chat users pick a model; background tasks resolve one through a deterministic router. Capabilities (tools, vision, structured output) are checked per provider.
Connector-agnostic	External systems plug in through a generic <code>Connector</code> interface or as user-added MCP servers, and the integration layer is generic. Connector-specific behaviour (for example list-to-detail drill-down pairs) is supplied through configuration. A few model-facing prompt strings still reference one knowledge-graph server's tools; their cleanup is tracked.

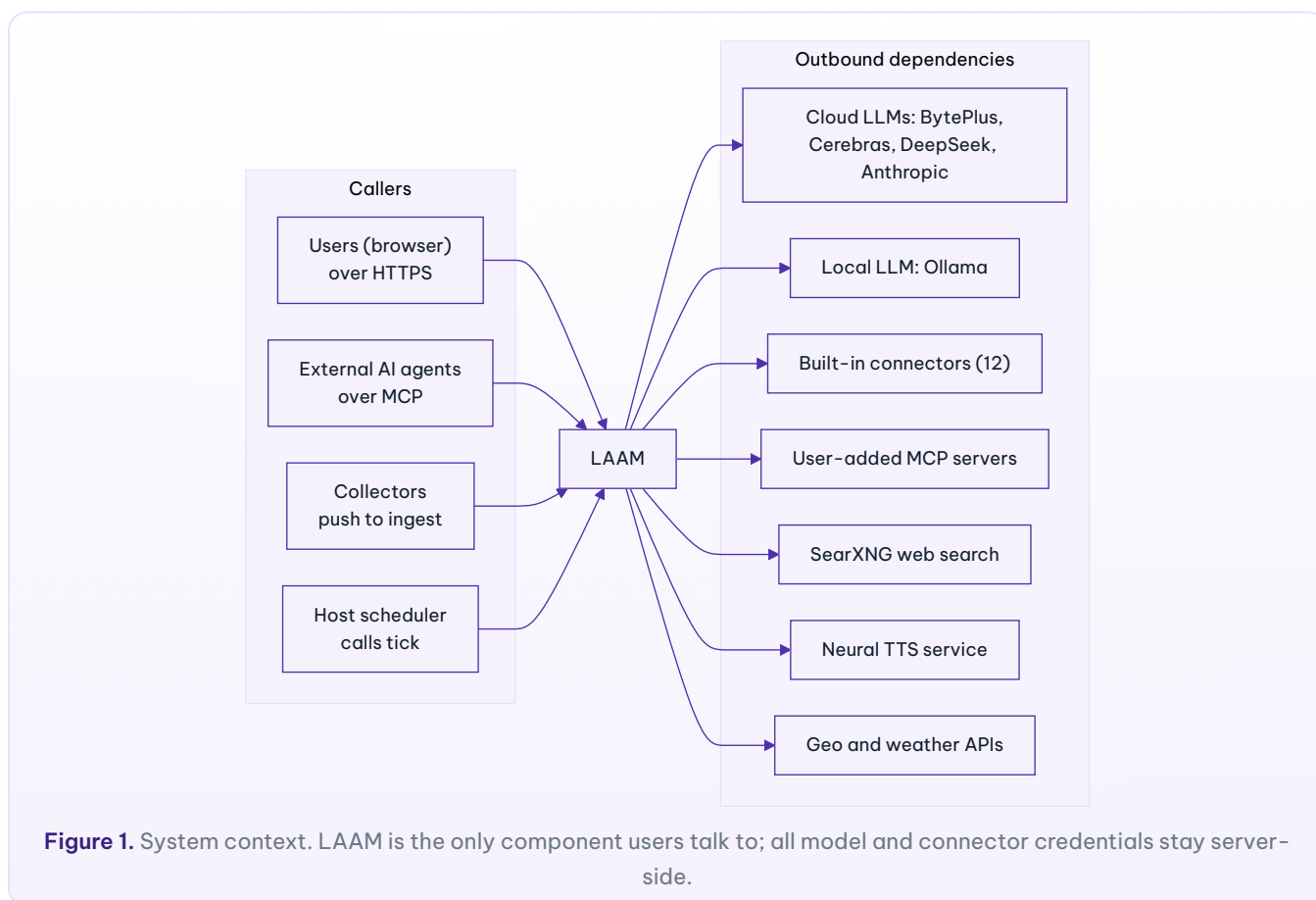
No code execution by the agent

A test in the unit suite fails if `process spawning`, `eval`, `new Function` or the `vm` module appear anywhere in the agent or workflow engines; there is no CI yet, so the suite is run by hand. The internal tools and the workflow node kinds are fixed in code, and the test also checks tool names and count. The assistant can only act through declared tools.

2 System context

LAAM runs as one application container on a customer-controlled host, next to its database and two optional sidecar services. Everything the assistant knows about the outside world arrives through one of four channels: LLM providers, connectors (built-in or MCP), web search, and public geo and weather services. Users reach it from a browser on desktop or mobile.

External actor	Interaction	Credential
Users	Chat turns, voice turns, workflow authoring and runs, connector management, administration.	Email and password (bcrypt), JWT session
External AI agents	Call LAAM's nine read-only monitoring tools over MCP; every call is itself recorded as a monitored session.	Personal access token (api or mcp kind)
LLM providers	Chat completions, tool calling, structured output. Server-side keys only; never sent to the browser.	Operator API keys (environment)
Built-in connectors	Read and write through each provider's REST API.	Per-user token or OAuth grant, encrypted at rest
MCP servers	Tool discovery (<code>tools/list</code>), tool calls, server instructions.	Per-user bearer token and headers, encrypted at rest
Collectors	Push parsed coding-agent session metadata with short, pattern-redacted text snippets (never full transcripts) from developer machines.	Collector access token
Host scheduler	Pokes the workflow tick endpoint once a minute.	Shared secret (constant-time compare)



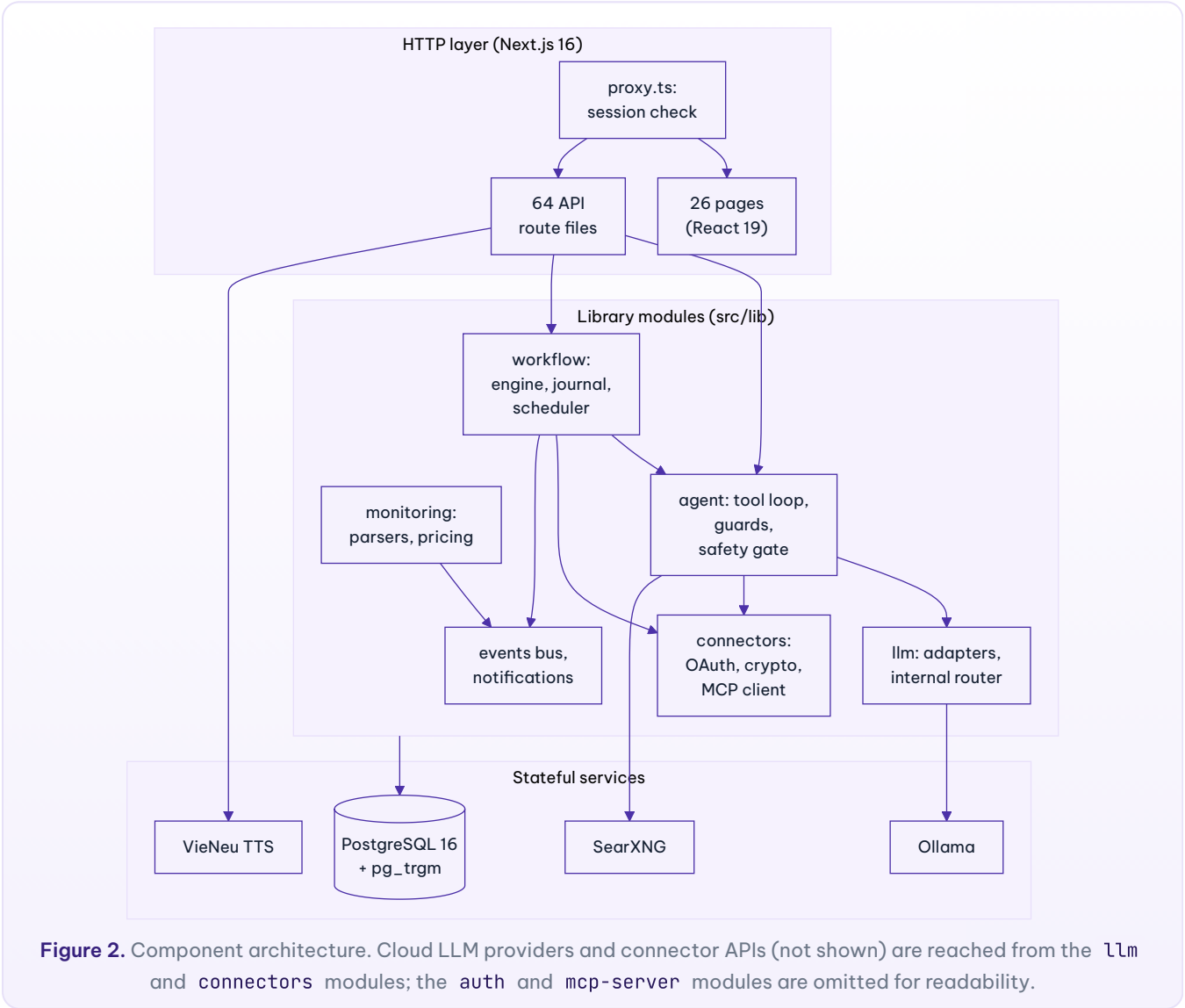
Deployment shape

The reference deployment is a Docker Compose stack on a single GPU-equipped host: the LAAM application container (multi-stage `node:24-alpine` image, Next.js standalone output, non-root user,

health check; OCR and PDF binaries bundled), the TTS service, SearXNG and a database admin UI. PostgreSQL 16 is an external instance shared with other services on the host network. Ollama and a small host-metrics agent run natively on the host so they can reach the GPU. Ingress is through a private mesh-VPN, optionally published to the internet through the same VPN's tunnel feature. Infrastructure, security and operations are covered in document 03 of this pack.

3 Component architecture

Inside the application, a thin HTTP layer (App Router pages and API route handlers) sits on a set of library modules under `src/lib`. The route handlers own authentication, request validation and streaming; the library modules own all behaviour and are unit-tested without a server.



Component	Responsibility	Location
Route protection	Edge-safe session check on every non-static path; a short public allow-list (landing, login, register, and three token- or secret-authenticated endpoints: ingest, MCP, workflow tick).	<code>src/proxy.ts</code>
Chat route	The chat turn: validation, conversation load, history budget, tool surface, prompt, provider branch, streaming frames, write suspension and confirmation.	<code>app/api/chat/route.ts</code>

Component	Responsibility	Location
Agent engine	Tool loop (<code>runToolRounds</code>), safety gate, guards and nudges, result panels and digests, context eviction, summarisation.	<code>lib/agent</code>
Provider layer	Adapters for Ollama, BytePlus, Cerebras, DeepSeek and Anthropic; the internal-model router; structured-output strategies.	<code>lib/llm</code>
Connector layer	12 built-in connectors behind one interface; OAuth flows and refresh under a Postgres advisory lock; AES-256-GCM credential store; generic MCP client with discovery, namespacing and SSRF guard.	<code>lib/connectors</code>
Workflow engine	Linear walker and parallel DAG scheduler, node executors, run journal, idempotency ledger, crash resume, cron scheduler, and the AI authoring pipeline.	<code>lib/workflow</code>
Auth	Auth.js v5 credentials login with JWT sessions refreshed from the database; RBAC helpers; login lockout and registration limiter.	<code>lib/auth</code> , <code>src/auth.ts</code>
Monitoring	Transcript parsers (shared with the collector), pricing table, stuck-session detection, unified read model over agent, chat and workflow runs.	<code>lib/monitoring</code>
Event bus	In-process publish/subscribe pinned on the global object; fans out to the SSE endpoint (<code>/api/events</code>) for sessions, workflow steps and per-user notifications.	<code>lib/events-bus.ts</code>
Boot hook	On start-up, marks workflow runs orphaned in <code>running</code> as <code>resumable</code> so the scheduler resumes them.	<code>src/instrumentation.ts</code>
Collector / host agent	Two zero-dependency Node scripts: the collector pushes parsed session metadata from developer machines; the host agent exposes CPU, RAM and GPU metrics for the voice surface.	<code>collector/</code> , <code>host-agent/</code>

■ Technology stack

Layer	Technology
Runtime and framework	Node.js 24, Next.js 16.2 App Router (standalone output), React 19.2, TypeScript 6 (strict), Tailwind CSS 4
Auth	Auth.js v5 beta (<code>next-auth 5.0.0-beta.31</code>) with Drizzle adapter, bcrypt password hashing, JWT session strategy
Data	PostgreSQL 16 with <code>pg_trgm</code> ; Drizzle ORM and Drizzle Kit migrations; <code>node-postgres</code> driver; Zod 4 validation
LLM	Anthropic SDK for Claude; OpenAI-compatible HTTP (no SDK) for BytePlus ModelArk, Cerebras and DeepSeek; Ollama HTTP API
MCP	Client: official <code>@modelcontextprotocol/sdk</code> (Streamable HTTP with SSE fallback). Server: hand-written JSON-RPC over Streamable HTTP
UI libraries	React Flow (workflow editor, session graph), Recharts, Leaflet with OpenStreetMap, three.js / react-three-fiber (voice visuals), react-markdown with sanitisation
Voice	Web Speech API (speech-to-text), Silero VAD in the browser, self-hosted VieNeu TTS (Python, FastAPI)

Layer	Technology
Documents	Tesseract OCR (Vietnamese, English, Simplified Chinese), Poppler PDF extraction, DOCX text extraction
Testing	Vitest, Testing Library, jsdom; a separate host-only eval configuration that calls real models

Single-process design

The event bus, rate limiters and workflow resume logic assume one application process. This keeps the stack simple for self-hosted deployments; horizontal scaling would move the bus and limiters to a shared store (for example Redis) first.

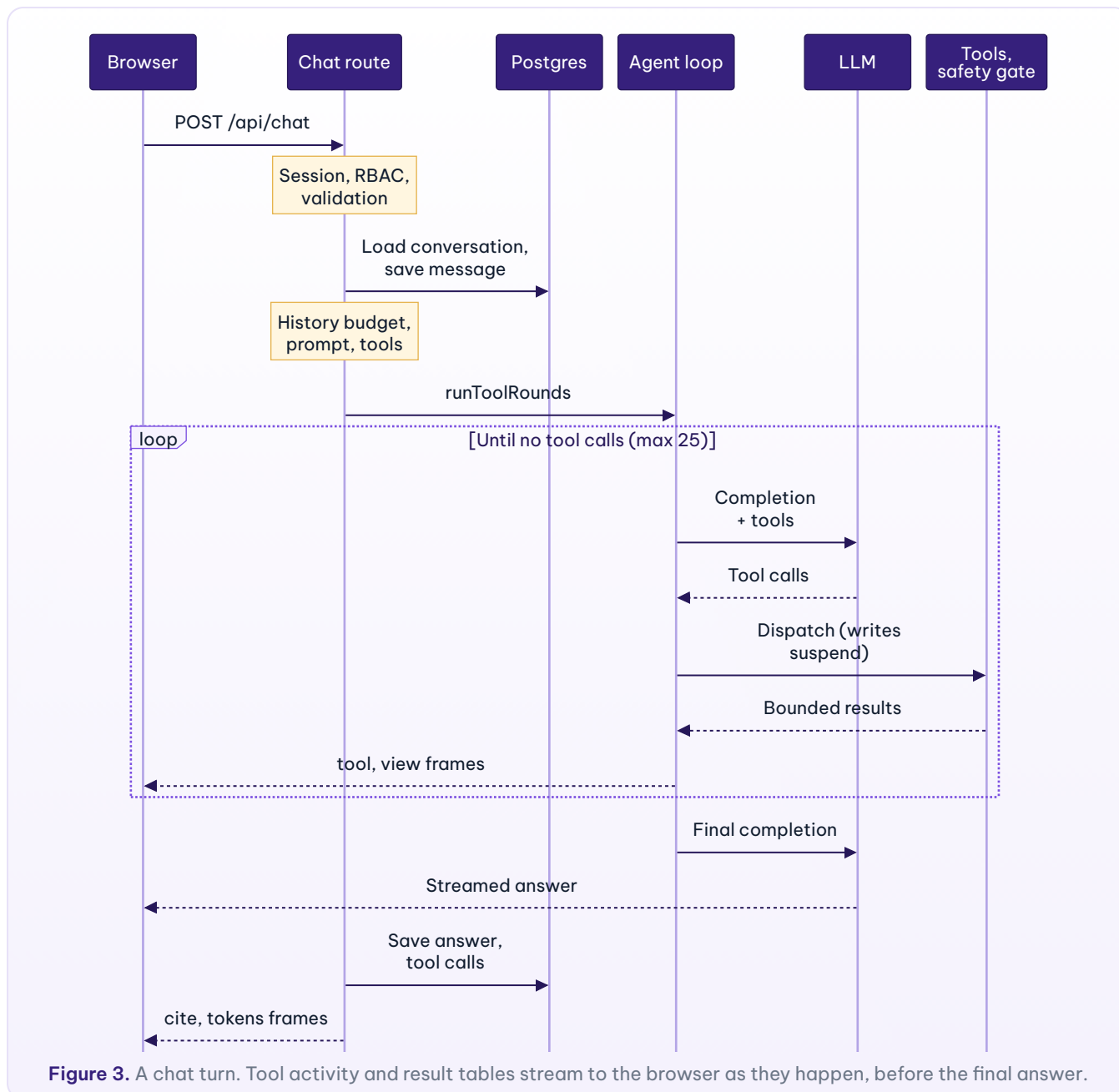
4 Chat and agent engine

A chat turn is one `POST /api/chat` request. The route authenticates the caller, assembles context, runs a tool loop against the chosen model, and streams the answer back while persisting every message and tool call. The same engine serves the voice surface, which only switches the output guide.

4.1 Turn lifecycle

The route performs these steps in order:

- 1. Authorise.** Require a session, then require a mutating role (viewers are read-only). Before any I/O, Claude model ids are checked against an allow-list (400 otherwise); BytePlus and Cerebras ids are recognised by list, and any other id goes to the local runtime.
- 2. Validate.** Reject empty messages, strip NUL bytes, cap images at two per turn and about 2.8 MB (base64) each.
- 3. Load context.** Load or create the conversation. A new conversation is stamped with its workspace after an ownership check; the workspace never changes afterwards.
- 4. Budget history.** Replay history within a provider-aware character budget. Overflow is folded into a rolling summary by the internal model (chapter 5) and stored on the conversation row.
- 5. Build the tool surface.** Internal tools, plus the user's connected built-in connector tools, plus every tool from the user's MCP servers, namespaced per server (chapter 6).
- 6. Pre-dispatch a quick tool** (optional). If the user picked a specific tool in the UI, code runs it deterministically before the model loop, through the same safety gate.
- 7. Assemble the system prompt** (section 4.2), then add proactive alerts (stuck or expensive monitored sessions, de-duplicated per conversation with a 6-hour cooldown) as a separate frame, never inside model prose.
- 8. Run the tool loop** (section 4.3), then the final completion (section 4.6).
- 9. Finalise.** Strip leaked tool-call JSON, persist the assistant message with token counts, persist every tool call and result, emit trailing frames.
- 10. Or suspend.** If the model proposed a write, the turn ends with a confirm card instead (chapter 7).



4.2 System prompt and persona precedence

The system prompt is assembled by code from six parts in a fixed order: base persona, today's date, a language hint (Vietnamese, English or Chinese), the tool block (only when tools exist), each MCP server's own instructions (captured from its `initialize` response, capped), and an output guide. The text guide teaches the table, chart and map block contract; the voice guide sets a spoken register, forbids reading identifiers aloud and allows at most one table or chart.

The tool block carries measured behavioural rules, phrased generically: a write must go through a tool and success must never be claimed without one; "search again" forces a fresh call; counts come from structured data; multi-metric comparisons are split into separate calls; natural-language data tools receive the user's own wording; SQL is never placed in a natural-language field.

Priority	Source	Effect
1	Explicit system-prompt override	Replaces the entire prompt; tool, date and language blocks and proactive alerts are dropped. Used for power users and internal paths.
2	Workspace persona	Replaces only the base persona; tool, date and language blocks remain. Resolved server-side from the conversation row, so a voice turn resumed elsewhere gets the same persona.
3	Custom agent selected by the user	Replaces only the base persona.
4	System default	Built-in assistant persona.

A workspace persona is either a custom agent or free-text instructions, never both (the API rejects both with 400). A deleted custom agent falls back to the default and the UI shows a warning badge.

■ 4.3 The tool loop

`runToolRounds` (`lib/agent/orchestrator.ts`) runs **until the model stops calling tools**. It replaced a fixed four-round cap in release 2.5.0, which had cut off any task needing more than three tool rounds (for example, summarising ten emails and then sending a report).

Mechanism	Behaviour
Runaway backstop	25 rounds by default (configurable, hard ceiling 50). The last round is forced to text.
Repeat guard	The same tool with identical arguments three times is treated as stuck (eight times for status-polling tools). The loop stops dispatching and tells the model to answer with what it has. "Stuck" and "out of rounds" are reported separately for operator tuning.
Context eviction	When the accumulated conversation exceeds a character budget (about 37k characters for local models, about 400k for cloud), the oldest tool results are replaced by a stub, keeping the three most recent. This follows the tool-result clearing pattern used by major model vendors.
Per-result processing	Each result is unwrapped, checked for emptiness, turned into a data panel if tabular, and annotated so the model does not retype a table already on screen (chapter 6).
One-shot nudges	At most once per turn each: read a page after a web search returned URLs; re-ask when the model answered from memory in round 0 despite having tools; stop using LAAM's own audit log for business questions. Workflow agent nodes disable nudges because their prompt is the instruction.

■ 4.4 Streaming protocol

A chat response is a `text/plain` stream, not Server-Sent Events. Plain answer text is interleaved with JSON control frames delimited by the ASCII record separator (U+001E), which never occurs in normal prose. The conversation id travels in a response header. Workflow runs, by contrast, publish over the SSE bus at `/api/events`.

Frame	Purpose
<code>tool</code>	A tool call started or returned (live trace in the UI).
<code>view</code>	A code-built data panel (table, optional chart hint) derived from a tool result.
<code>cite</code>	Sources used by the answer.

Frame	Purpose
pending_write	A write awaiting human confirmation: code-built preview and sealed token.
proactive	Alerts about stuck or costly monitored sessions.
tokens	Token usage, sent only when the provider actually reported usage (no fabricated zero cost).
hb	Heartbeat while a reasoning model is thinking, so idle proxies do not drop the connection.

4.5 Reasoning effort by phase

For reasoning models on BytePlus and Cerebras, LAAM sets reasoning effort separately for **tool rounds** and for the **final answer** (each falls back to a shared setting; only `low`, `medium` and `high` are forwarded). This lets a deployment trade latency against depth where it matters. The shipped example configuration sets the final phase to `low` and deliberately leaves tool rounds at the provider default: low effort during tool selection produced a wrong "nothing found" answer on a data-lookup question in testing. DeepSeek has its own settings, and its "thinking" mode is disabled on tool rounds after a traced 100-second-plus call.

4.6 Final answer and context management

- **Digest before the final completion.** Large tabular results reach the model as a digest (row count, columns, code-computed per-column totals across all rows, a few sample rows). The raw rows stay in `chat_tool_call`, so a reloaded conversation rebuilds its tables exactly.
- **Question pin.** After a tool-using turn on the cloud (BytePlus or Cerebras) path, the user's question is restated before the final completion. It was added after a measured case where the answer drifted to an earlier topic.
- **Empty-answer recovery.** If a cloud model's final stream is empty, the engine retries exactly once with an "answer now from what you have" nudge; if still empty, it sends a coded notice. A blank bubble is never shown.
- **Rolling summary.** LAAM has no separate long-term memory store. Conversation memory is the rolling summary on the conversation row plus the persisted tool turns. Any shared memory a customer wants comes through a connected MCP server like any other tool.

5 Model provider layer

LAAM talks to five model providers through one adapter contract. Users choose among chat models in a picker; background tasks (summaries, workflow agent nodes, AI authoring) have no picker and resolve a model through a deterministic router.

Provider	Transport	Chat picker	Tool loop	Vision	Notes
Ollama (local)	Ollama HTTP API	Always	Yes	Yes (≤ 2 images)	Zero marginal cost; runs on the host GPU.
BytePlus ModelArk	OpenAI-compatible HTTP	When key set (preselected)	Yes	Not wired	Eight whitelisted models, default <code>gpt-oss-120b</code> .
Cerebras	OpenAI-compatible HTTP	When key set	Yes	Not wired	Shares the BytePlus code path.

Provider	Transport	Chat picker	Tool loop	Vision	Notes
DeepSeek	OpenAI-compatible HTTP	No	Yes	No	Background tasks only.
Anthropic Claude	Anthropic SDK	When key set	No	No	Text-only chat; asking it to drive tools fails loudly.

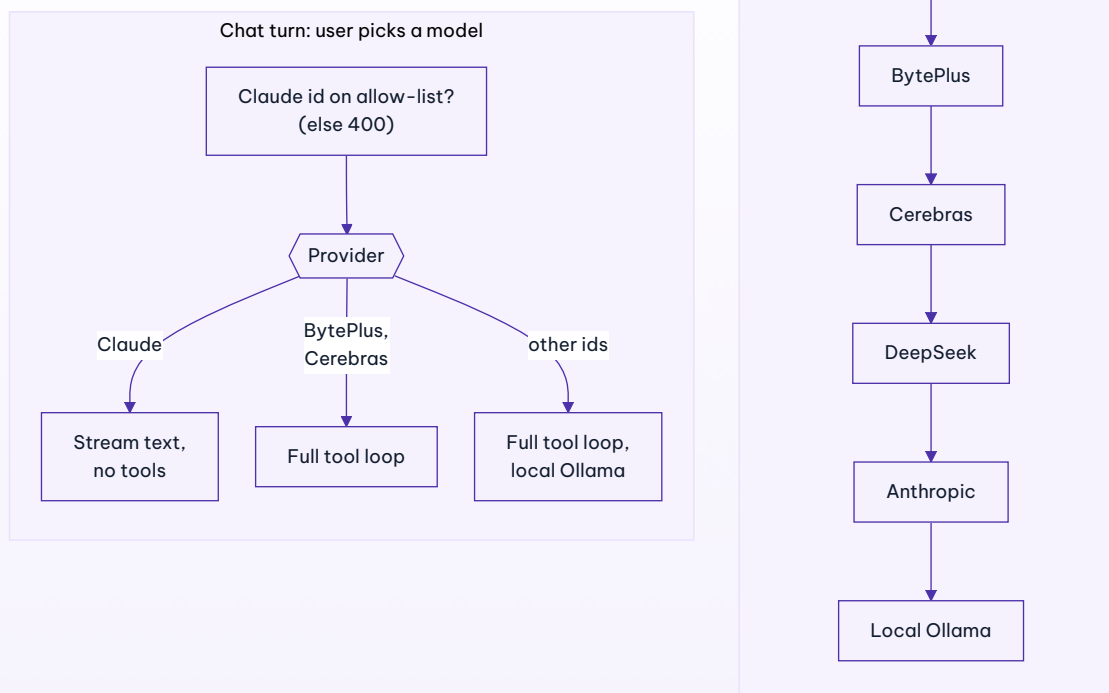


Figure 4. Chat routing and the internal-model router. The router resolves exactly one model per call; the first configured tier wins.

• Routing rules

- **Cloud-first default.** The chat picker preselects BytePlus if configured, then Cerebras, else the local model. A deployment without cloud keys runs entirely on local models.
- **No silent cross-provider fallback.** A provider error produces a coded, tri-lingual notice (authentication, rate limit, overloaded, connection, API) and the turn is persisted. BytePlus and Cerebras retry once on 429, 503 or 529 after a configurable delay.
- **Three internal entry points.** `callModelText` (no tools), `callModelChat` (tool loop) and `callModelGenerate` (structured JSON).
- **Capability checks.** Claude can summarise but cannot run a workflow agent node; the run fails at that node with an explicit error rather than producing an untooled answer.

• Structured output per provider

Workflow authoring needs reliable JSON. Support differs by provider and model, so the strategy is chosen per provider from measurements: JSON mode where supported; prompt-only JSON when a model rejects the `response_format` parameter; a forced tool call ("structured tool") as the default for

branch derivation on BytePlus `gpt-oss-120b` (10 of 10 in testing); strict JSON schema on Cerebras. DeepSeek's strict function calling measured worse and is opt-in only.

■ **Cost accounting**

A pricing table (USD per million input and output tokens) covers Claude, BytePlus, Cerebras and DeepSeek models. An unmatched model is costed at a default Sonnet-level estimate; models whose price is not published show no cost rather than a guess. Tokens are recorded per chat message and per workflow step together with the model that actually answered, and cost is derived at write time. Monitoring shows model, tools and cost for chat and workflow runs alongside monitored agent sessions.

6 Tools and connectors

Everything the assistant can do is a tool. Tools come from three sources: a closed set of internal tools, the built-in connectors a user has connected, and the MCP servers a user has added. All three share one dispatch path and one safety gate.

■ **6.1 Internal tools (always available)**

Group	Tools
Monitoring (9, read-only)	<code>laam_list_agents</code> , <code>laam_get_agent</code> , <code>laam_query_stats</code> , <code>laam_list_machines</code> , <code>laam_find_stuck</code> , <code>laam_search_sessions</code> , <code>laam_get_timeline</code> , <code>laam_query_audit</code> , <code>laam_metrics_digest</code> . They read LAAM's own monitoring store, not business data. The same nine are exposed to external agents through LAAM's MCP server.
Web (2)	<code>web_search</code> over self-hosted SearXNG (fails soft if the service is down); <code>web_read</code> fetches a URL as text, checking the host of the URL and of every redirect hop against a blocklist of private and metadata hostnames, capped at 6,000 characters.
Utility (1)	<code>util_calc</code> : deterministic arithmetic via a shunting-yard parser, no <code>eval</code> .

■ **6.2 Built-in connectors**

Each built-in connector implements the same `Connector` interface: an id, an auth mode, and a list of tools that each declare `kind: "read" | "write"` and whether they are safe to run unattended in a workflow. Credentials are encrypted per user (AES-256-GCM with a key derived per user by HKDF-SHA256). OAuth token refresh runs at the single `execute()` chokepoint under a Postgres advisory lock, because several providers rotate single-use refresh tokens; a revoked grant flips the connector to "needs reconnect".

Connector	Tools	Writes	Auth	Write examples
GitHub	8	2	Token	Create issue, comment
Trello	7	3	Token, optional one-click authorise	Create / update card, comment
Jira	6	2	Atlassian OAuth 3LO or API token	Create issue, comment
Google Drive	5	1	Google OAuth	Create folder

Connector	Tools	Writes	Auth	Write examples
Google Calendar	4	1	Google OAuth	Create event
Gmail	4	1	Google OAuth	Send (confirm in chat; recipient allow-list in workflows)
Slack	3	1	Slack OAuth v2	Send message
WhatsApp	1	1	Cloud API token	Send message
Zalo	2	1	Zalo OA OAuth v4 (PKCE)	Send message
APILayer catalog	2	0	One access key	– (9 catalogued APIs)
Public-APIs catalog	2	0	None	– (8 hand-verified keyless APIs)
Demo	2	1	None	Create task (offline write-gate demo)
Total	46	14		

The two catalog connectors are SSRF-safe by construction: the model searches a hand-authored catalog and calls an entry, and each entry pins its own URL template, so the model can only request URLs LAAM wrote. OAuth flows are hidden until the operator configures the provider's client credentials; Jira also accepts an API token.

■ 6.3 MCP client – the generic integration path

Any system that speaks the Model Context Protocol can be added by a user at runtime: a database gateway, a knowledge-graph server, a SaaS vendor's MCP endpoint. The MCP path is generic; a few model-facing strings still name one knowledge-graph server's tools (cleanup tracked). Server configuration (URL, bearer token, headers) is stored in the same encrypted per-user credential table as built-in connectors.

Property	Implementation and rationale
Transports	Streamable HTTP first, one retry over SSE for older servers. Connections are per operation and always closed.
SSRF guard	At connect time the server URL's hostname is resolved and every resulting address is checked against private, link-local, metadata, IPv6 unique-local and IPv4-mapped ranges before a transport opens. Hardening item before production: one DNS-pinned guard for all outbound fetches, adding the carrier-grade NAT range and a re-check on redirect.
Namespacing	Each tool is exposed to the model as <code>mcp__<server>__<tool></code> , so servers cannot collide with each other or shadow built-ins. Server slugs stay stable when a server is renamed or re-pointed.
Prefix de-collision	When one tool name is a strict prefix of another on the same server, the shorter name is padded. Measured on one provider: the longer tool was reached 0 of 6 times before the fix and 6 of 6 after; on one 49-tool server it made 11 tools reachable.
Server instructions	Each server's <code>initialize</code> instructions are captured and placed in the system prompt, so scoping information a server publishes is not re-derived with extra list calls on every question.

Property	Implementation and rationale
Limits	200 tools per server (logged truncation), 15-second call timeout, error text to the model capped at 300 characters, 200k-character raw backstop with a visible truncation marker; per-result shaping to 8k characters (local) or 120k (cloud).
In-band errors	MCP's <code>isError</code> flag is carried through: the workflow runtime fail-stops on it and AI authoring discards such observations.

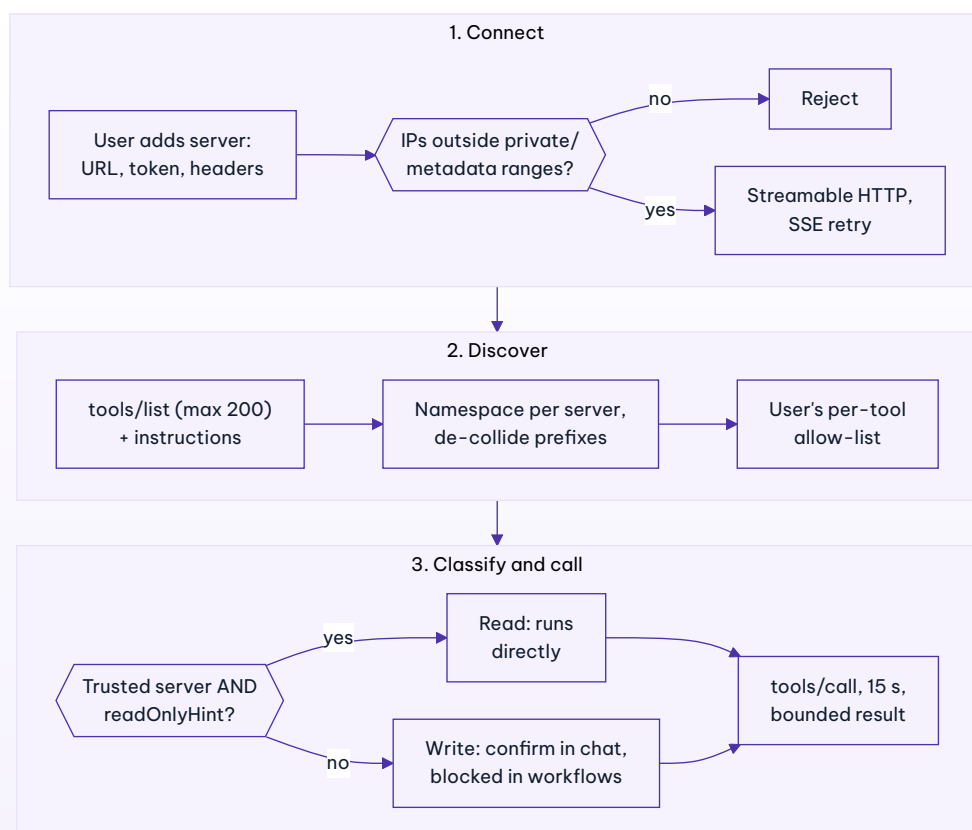


Figure 5. MCP tool path. Classification fails closed: a tool is a read only when both the user and the server say so.

■ 6.4 Result handling: code-built tables and digests

MCP results arrive as text containing JSON, while internal tools return objects. A single unwrapping function normalises both, and every shape-inspecting step uses it; a guard that skipped it would silently never fire.

- Data panels.** When a result is tabular (at least 3 rows and 2 columns with a consistent key set, up to 500 rows), code builds a table and streams it as a `view` frame; a chart hint is added for up to 25 rows. The model is then told the table is on screen. The motivation was measured: on 13 test questions, only 2 results met the old size threshold for a panel; in the other 11, the numbers on screen were the model retyping the result, and it introduced errors (a truncated product name, a spurious space in an identifier, a lost cell and a repeated row).
- Digests for the model.** Large results reach the model as counts, columns, code-computed totals and sample rows. In one measured case a 62-row result took 103 seconds and 3,225 output tokens to retype, with errors; the digest removes that cost.
- Formatting rules.** Markdown tables are repaired by code; numbers use thousand separators and explicit currency units.

7 Reliability guards

Language models confabulate, pick near-miss tools and give up early. LAAM does not rely on prompting alone to prevent this. Each failure mode observed in testing has a structural guard in code, triggered by observable facts (a missing tool call, an empty result, a malformed name) rather than by classifying the user's intent.

7.1 Write confirmation round-trip

The write gate is the most important guard. Every dispatch passes through `withSafety`, which classifies the tool from its own declaration. Unknown tools are treated as writes. In chat, an unconfirmed write never executes: it raises a signal that the route turns into a suspended turn.

Three design choices matter here. The preview shown to the user is built by code from the arguments, never from model prose, so the user approves what will actually be sent. The token is stateless (sealed with authenticated encryption) and carries the original model, so the narration after confirmation continues in the same voice. And the token's nonce is checked against the audit log before dispatch, so a confirm that has already been recorded is rejected. Hardening item before production: an atomic nonce claim and idempotent retry for confirmed MCP writes.

7.2 Grounding guards

Guard	Trigger	Why it exists
Grounding re-ask	Round 0, tools available, model answered with no tool call. Re-ask once (with an escape for small talk).	A model invented a complete project record, including a non-existent owner, instead of looking it up.
Empty is not absent	A tool returned zero rows or a zero count.	On an anomaly query, one phrasing returned nothing while three other phrasings returned 9 to 18 rows. The model is told an empty result is not proof of absence, shown the query it used, and allowed one re-ask in the user's own words.
Write-claim guard	User text looks like a write request; the answer claims success; no write tool executed.	Local models narrated writes they never made. When write intent is detected, output is buffered and unsupported success claims are replaced.
Tool-intent reroute	The arguments fit exactly one other tool.	A model called a tool whose name was a prefix of the intended tool, with the intended tool's arguments.
Tool-name repair	Formatting tokens leaked into the tool name.	An open-weight model's chat format occasionally leaked channel markers and suffixes into tool names.
Question pin	A tool-using turn on the cloud (BytePlus or Cerebras) path.	An answer drifted to the previous question after a long tool sequence.
Artifact strip	Tool-call JSON at the head of the answer.	Persisted answers occasionally began with raw JSON.
Audit-misuse nudge	LAAM's own audit log queried for a business question.	The model read the wrong log and concluded "no data".

Guard	Trigger	Why it exists
Data-fetch guard PLANNED	Only structure or listing tools were called. Flag-off by default; configured per deployment.	A model inspected schemas repeatedly, then answered "no data" without fetching any.
List-to-detail drill-down PLANNED	After a configured list tool, the question names exactly one entity from its result. Flag-off by default.	Code calls the paired detail tool. Pairs are configuration, so no connector name is hard-coded. In testing, 12 of 12 voice detail questions then reached the detail tool.

Guards are one-shot per turn and fail soft: none of them can block an answer permanently, and the write signal always propagates through them.

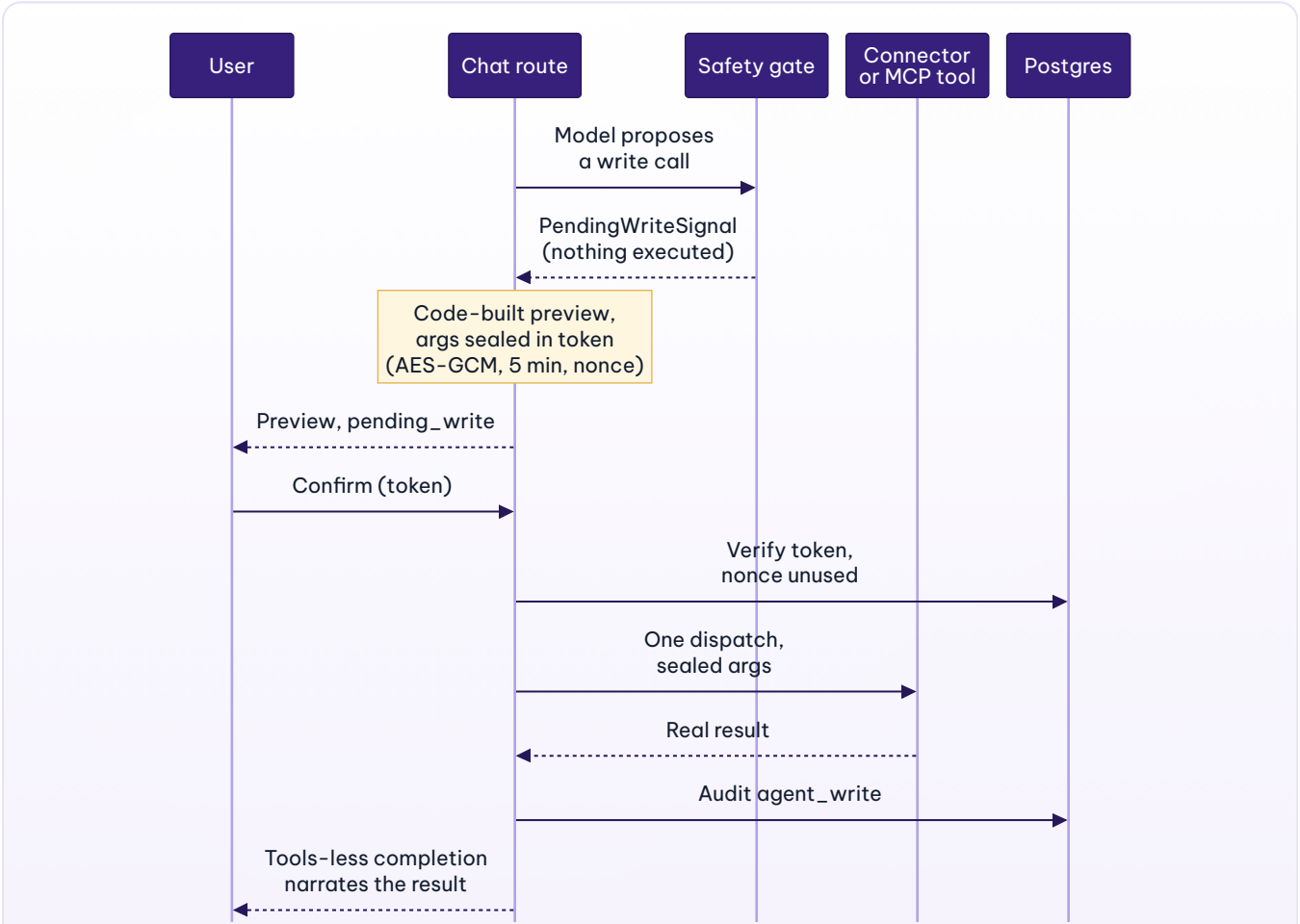


Figure 6. Write confirmation in chat. The confirmed call uses the sealed arguments, not a fresh model output, and the narrating completion has no tools, so it cannot start another write.

8 Workflow engine

A workflow is a JSON graph of typed nodes stored per user. It can be run on demand, on a cron schedule, or as a dry-run test. Runs are journaled step by step so they survive a process restart, and writes are recorded in an idempotency ledger so a resumed run never sends twice.

■ 8.1 Graph model

Node kind	Behaviour
agent	Runs the tool loop with LAAM's internal read tools only, then one final call through the internal router. Optional custom-agent preset (a missing preset fails loudly) and optional output schema. An <code>extract</code> mode pulls a field from an earlier step in code, with no model call; it was added because model-based extraction was the entire cost of some runs (about 3 s per call against 24–55 ms MCP calls).
connector	Calls a built-in connector action with templated arguments (<code>{{steps.x.output}}</code> , <code>{{trigger}}</code>).
mcp	Calls a tool on a user's MCP server. An optional <code>pollUntil</code> re-issues a status call until a field reaches a value; it is inferred by code from observed calls, never authored by the model.
condition	Evaluates a predicate (10 operators, combinable with <code>all</code> / <code>any</code>) and follows the <code>true</code> or <code>false</code> edge.
foreach	Runs a sub-graph once per item with <code>{{item}}</code> and <code>{{index}}</code> , sequentially.

External data enters a workflow only through connector and MCP nodes. The node kinds are fixed in code. Budgets: 200 steps and 100 for-each items per run; in parallel mode, fan-out of at most 12 edges per node.

■ 8.2 Execution modes

- **Linear** (default): a recursive edge walker evaluates conditions, runs for-each bodies, shares one budget counter and stops at the first failure.
- **Parallel** (per-graph flag): a topological DAG scheduler with fan-out and OR-join fan-in, multiple start nodes, and tri-state pruning of untaken branches. Two semaphores separate model-bound work (2 concurrent) from I/O (6 concurrent). Fail-stop, cancel and budget are cooperative: an abort flag, then a drain of in-flight nodes before finalising. A deadlock detector guards the scheduler, and a golden test proves that a linear graph at concurrency 1 produces a byte-identical run to the linear walker.
- **Validation** uses Kahn's algorithm for cycle detection (a previous depth-first check falsely flagged diamond shapes), reachability across start nodes, and a family of lint rules that reject references a node cannot actually produce. The same validation gates Save.

■ 8.3 Run lifecycle, durability and idempotent writes

- **Snapshot.** The graph is copied onto the run row at start, so editing a workflow never changes a run in progress.
- **Journal.** Each node writes a `workflow_run_step` row (input, capped output, tokens, cost and the model actually used) and publishes a step event with a 280-character output preview for the live canvas.
- **Idempotent writes.** Before a write, the executor claims a row keyed on (run, node, iteration) with a unique constraint. After success the row is marked done with the output. On resume, a done write replays its stored output; a claimed-but-unrecorded write fails loudly instead of risking a duplicate email or message.
- **Crash resume.** On boot, runs still marked running become resumable. The next scheduler tick claims up to 25 of them with `FOR UPDATE SKIP LOCKED` , rebuilds context from the journal and skips completed nodes.

- **Output gates.** An agent node with an output schema is checked for missing required keys (a null counts as missing) and for type mismatches; it gets one self-repair attempt with the exact field names, then fails at that node. The type check exists because a presence-only check once let an empty result list pass and a run finished all-green on no data.
- **Recipient gate.** Writes that leave the organisation (email, Slack, WhatsApp, Zalo) require every resolved recipient to be in that channel's allow-list; an empty allow-list blocks. The recipient parser rejects header injection.

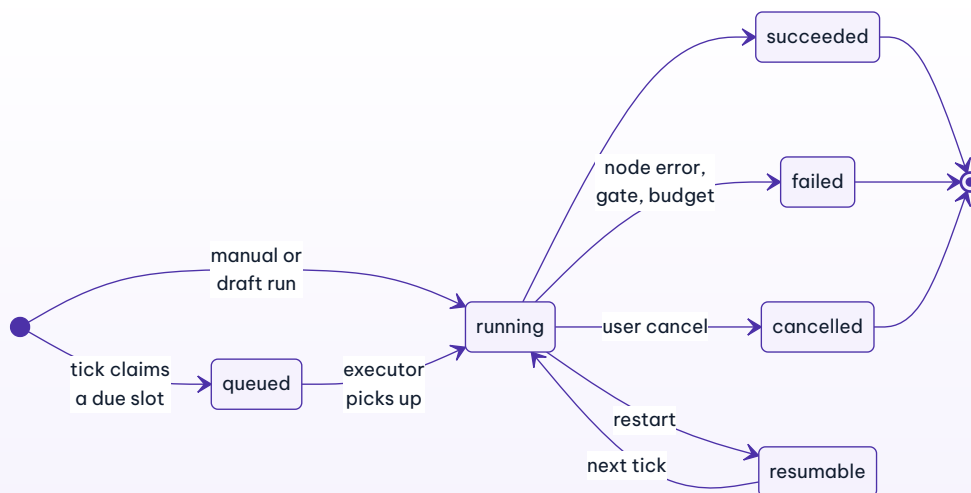


Figure 7. Workflow run states. A run interrupted by a process restart becomes resumable and continues from its journal on the next tick; a cancelled run is never resumed.

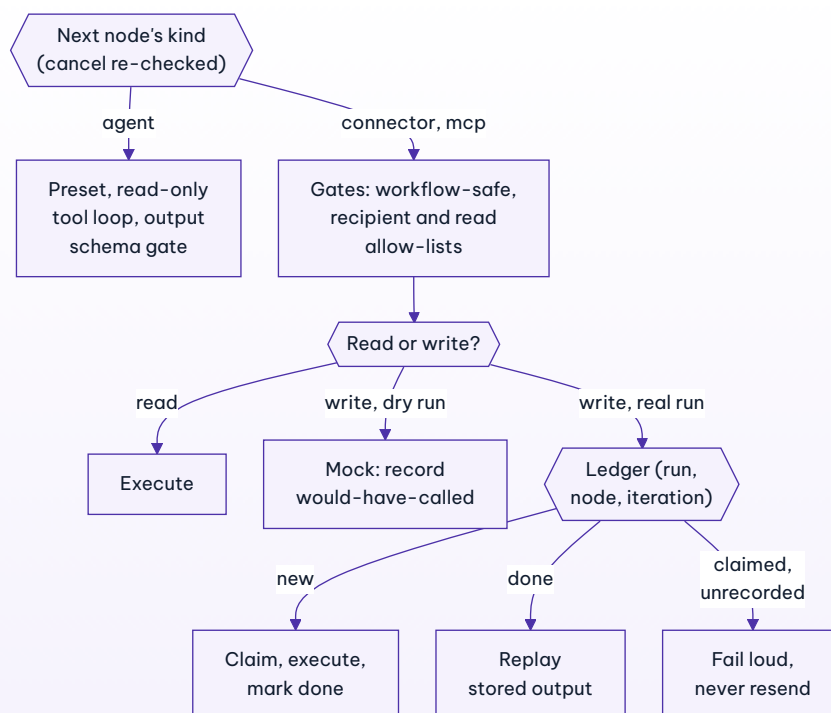


Figure 8. Per-node execution. Every step is journaled; every write goes through the idempotency ledger.

8.4 Triggers and scheduler

Trigger	Path
manual	POST /api/workflows/[id]/run , optionally {dryRun:true} : reads run for real, writes are mocked.

Trigger	Path
draft	The same endpoint with an unsaved graph from the AI authoring panel. Accepted only with <code>dryRun:true</code> , then normalised and validated like a saved graph.
schedule BUILT	Five-field cron per schedule. <code>POST /api/workflows/tick</code> claims due schedules atomically (insert the run and advance the next-run time in one transaction, unique per schedule slot), then executes queued and resumable runs. Missed slots fire once and are counted rather than back-filled.

The tick endpoint is driven by an external host scheduler once a minute and authenticated with a shared secret (constant-time comparison); there is no in-process timer. Runs can be cancelled at any time; the engine sees the cancel at its next pre-node check. Terminal states raise an in-app notification with a deep link. Six localised templates (overnight digest, stuck-agent alert, scheduled triage, judge-verified digest, multi-source report to email, and a demo) and a save-as-template action are included.

9 AI workflow authoring

BUILT Users can describe an automation in plain language and have LAAM build it. The authoring pipeline is **trace-grounded**: the model must first observe real tool calls on the user's own connectors, and it may only build a graph out of calls it has actually seen succeed. Code, not the model, compiles the final graph.

Stage	What it guarantees
Probe	The model is shown parameter names but never real values, so it cannot copy a live identifier into a saved graph (an earlier bug did exactly that). Writes are blocked and recorded, never executed.
Plan DSL	The plan schema cannot express a tool name, connector id or literal argument value. Plan nodes reference observed calls by opaque ref and wire arguments to earlier steps.
planGuard	Rejects on the first failure, naming the real alternatives so the error doubles as the repair prompt: the ref must exist; argument keys must be real; values must be templated, not bare literals; sources must be grounded; extracted fields must exist in the observed payload; fan-out ≤ 12 ; no unused reads.
Data closure	Pulls in the producer calls a plan depends on, including calls from earlier turns of the authoring conversation. Without it, "the first item" was frozen to one specific item forever.
Compile	High-entropy literals become step references; recipient and body arguments of writes are stripped and re-asserted as code-owned; parallelism is inferred.
Merge	Human-confirmed write nodes (id, arguments, recipient) stay invariant across rebuilds; hand-edited prompts are preserved when the step list only extends.
Verify	Runs the graph for real with mocked writes (40 steps, 2 for-each items, 60-second deadline) and badges failing nodes on the canvas.

Two authoring surfaces

- **One-shot generate** (`POST /api/workflows/generate`): probe, plan, verify; the editor applies the result as an undoable proposal behind a confirm card, even if some nodes are red, so the user sees exactly what failed.

- **Multi-turn authoring panel** (`POST /api/workflows/generate/chat`): each turn runs a write-blocked tool loop, then overlaps a prose reply with the plan build (saving a median 2.6 s per turn). A no-data gate refuses to paint a graph if no read actually resolved to an answer and asks a clarifying question instead. Only a verified green graph is offered, with up to three observed sample rows. Writes are appended in a separate, human-confirmed step. A **test run** executes the proposed graph as a **draft** run and digests the outcome back into the conversation with identifier-shaped values masked, so the model can react to real outcomes (for example add a branch for a "clarification needed" status) without hard-coding values.
- **Review** (`POST /api/workflows/review`): a plain-language review of the graph currently in the editor.

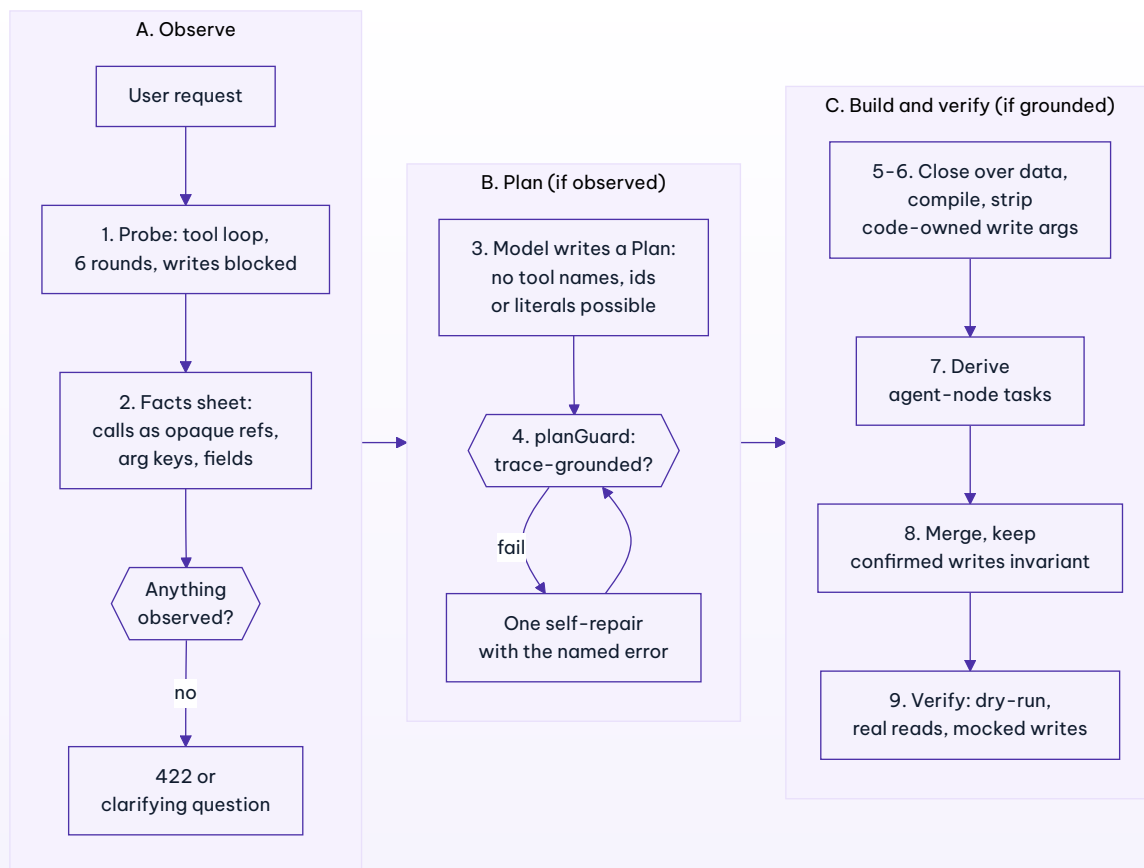


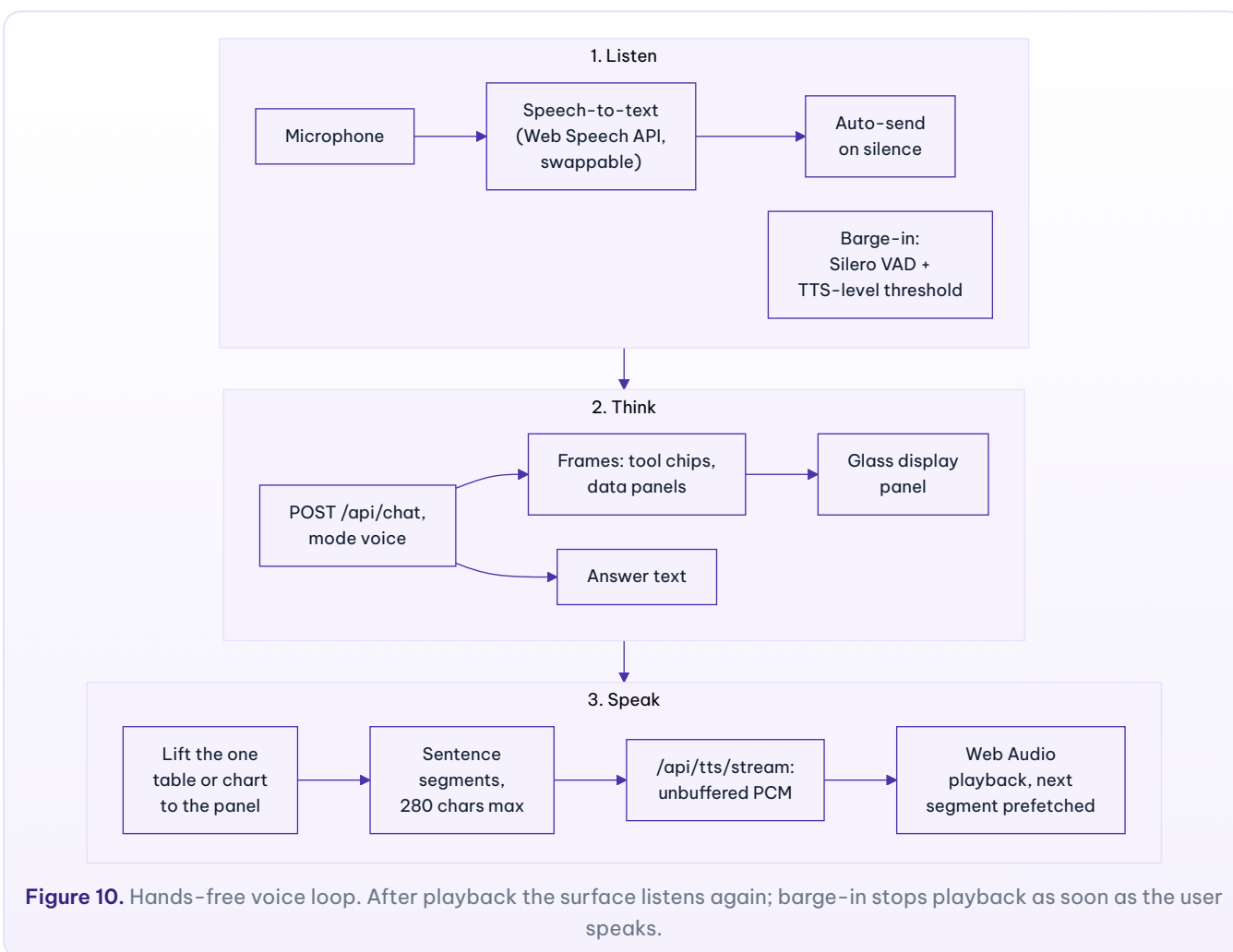
Figure 9. Trace-grounded authoring. If the repaired plan still fails, the pipeline falls back loudly to an older trace compiler and says so.

10 Voice pipeline

The voice surface is a full-screen command centre that runs the same chat engine hands-free. Custom agents sit on an inner ring and tool groups on an outer ring; disconnected connectors are dimmed. Speech in and out is streamed so that the assistant starts talking before the whole answer is synthesised.

- **Same engine, different guide.** Voice turns use the chat route with `mode: "voice"`. The voice guide asks for a spoken register without identifiers, and states that brevity must not reduce lookup depth. It targets a measured problem: 3 of 17 voice turns were shallow or fabricated, against 0 of 6 in text mode. In a 12-turn retest, no voice turn skipped tool calls; 2 were still shallow because the model chose the wrong tool, which the (flag-off) list-to-detail drill-down addresses.

- **Speech-to-text.** A swappable provider interface; the first implementation is the browser's Web Speech API. A self-hosted Whisper provider is **PLANNED**.
- **Barge-in.** Two gates: a voice-activity model served from LAAM itself, and a dynamic threshold tied to the current playback level so the assistant's own voice does not trigger an interruption.
- **Text-to-speech.** A self-hosted VieNeu TTS service (Python, CPU) with a Vietnamese preset voice and a cloned English voice. The streaming endpoint pipes 48 kHz PCM unbuffered; sentence segmentation keeps every synthesis request well inside the route timeout regardless of answer length, and the next segment is fetched while the current one plays. The container's CPU cap is paired with matching thread limits; capping CPUs alone collapsed throughput. The browser's built-in speech synthesis is the fallback.
- **Display panel.** The model may include at most one table or chart per voice answer; it is lifted out of the spoken text and shown on a non-modal panel. Result tables are collapsed by default on both chat and voice surfaces.



11 Data model

LAAM stores everything in one PostgreSQL 16 database: 22 tables and one enum, managed by 19 versioned Drizzle migrations (migrations only; schema push is never used against real data). The model is a single organisation with roles, not multi-tenant; per-user data is isolated by foreign keys to **user** and ownership checks in every query.

11.1 Identity and governance

Access tokens are high-entropy random strings stored as SHA-256 hashes and shown once. One table serves three kinds: collector tokens (bound to a machine), personal API tokens and MCP tokens. Disabling a user is one transaction: set `disabledAt`, revoke every token, write an audit entry. Notifications de-duplicate on a partial unique index over (user, dedupe key). Notifications de-duplicate on a partial unique index over (user, dedupe key).

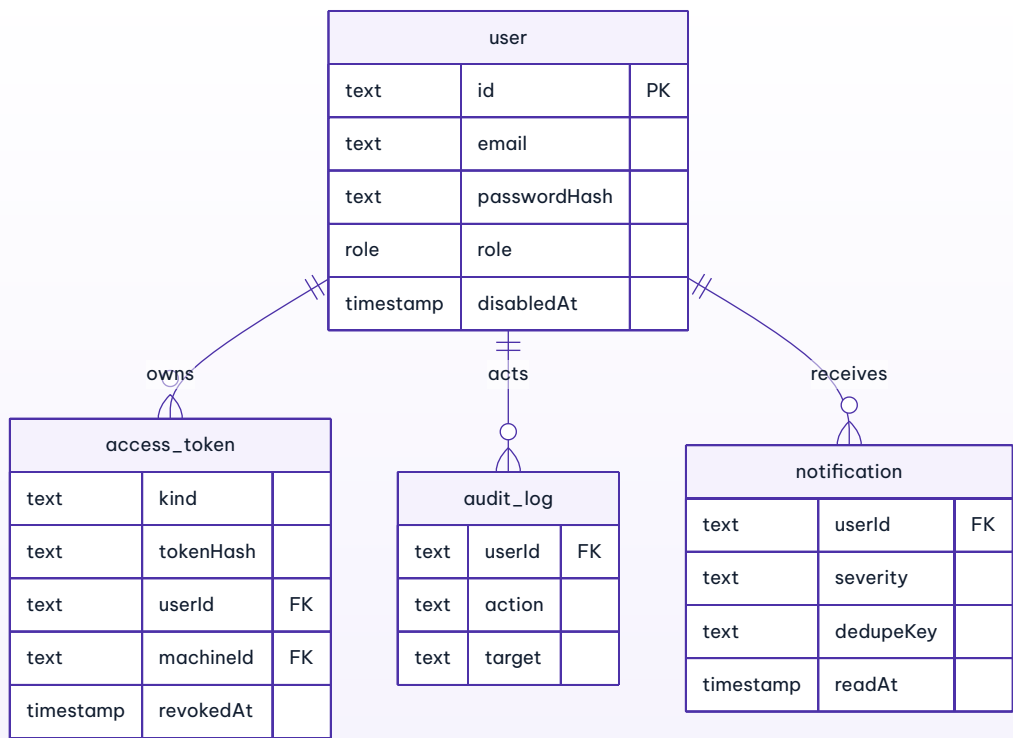


Figure 11. Identity and governance. Login uses JWT sessions; the Auth.js adapter tables (`account` , `session` , `verificationToken`) exist for compatibility and are not shown.

11.2 Chat, agents and connectors

`connector_credential` holds one encrypted blob per (user, connector) and also stores each user MCP server's configuration under a `mcp:<slug>` key. Blobs are encrypted with a per-user key, so a blob copied to another user's row does not decrypt. Custom-agent references from workspaces and workflow nodes are deliberately soft (no foreign key) so that a deleted preset is detected and reported at run time rather than cascading.

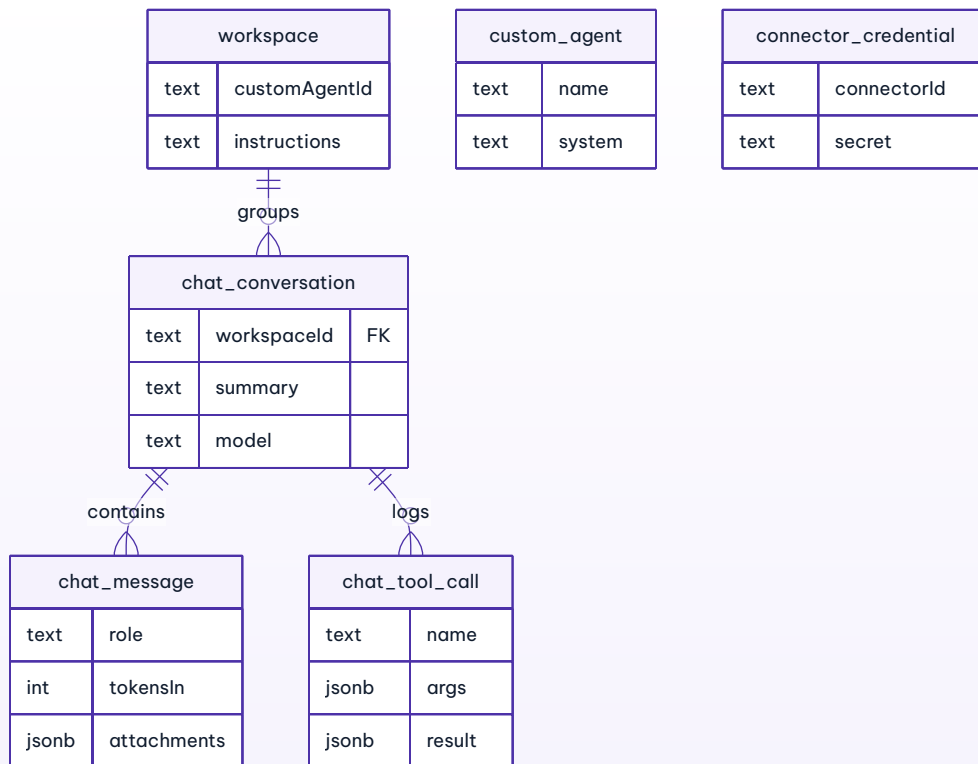


Figure 12. Chat, agents and connectors. Workspaces, conversations, custom agents and connector credentials each belong to one `user` (figure 11). Raw tool results are kept in `chat_tool_call` so tables can be rebuilt on reload.

11.3 Workflows

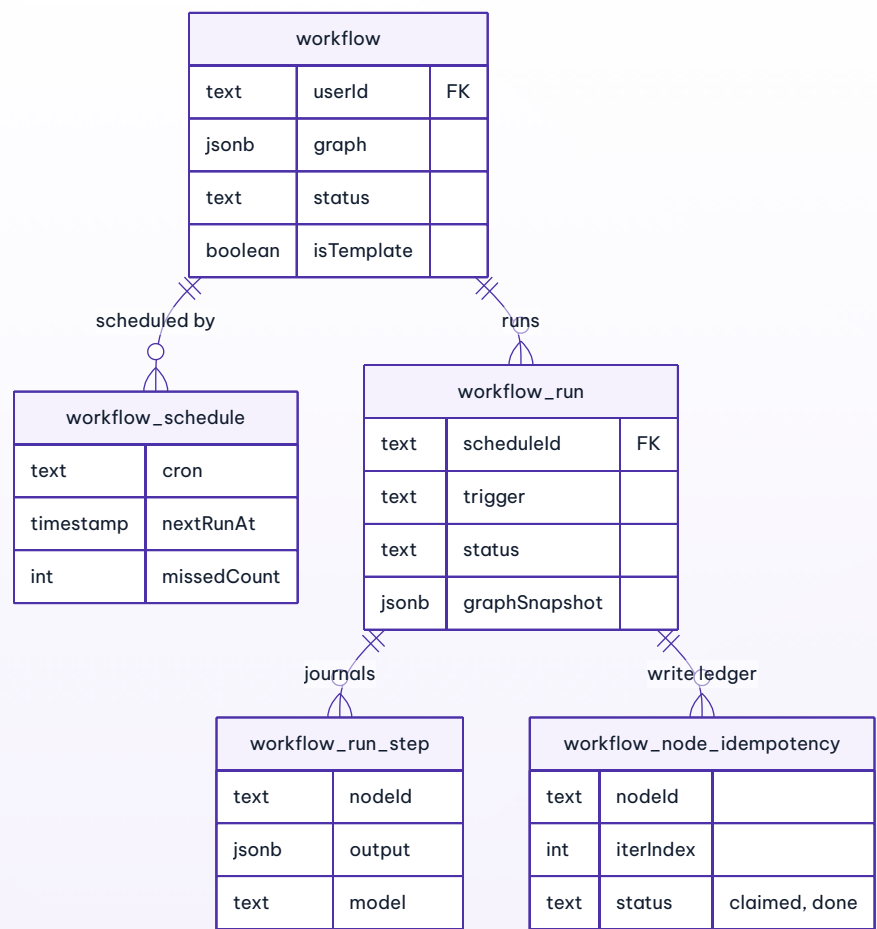


Figure 13. Workflows. A scheduled run references its schedule through `scheduleId`. Unique constraints on (schedule, scheduled-for) and on (run, node, iteration) make schedule claims and writes idempotent.

11.4 Monitoring and evaluation

Agent sessions derived from coding-agent transcripts are shared across the organisation by design; sessions created through LAAM's API or MCP server are scoped to their principal. Each machine authenticates its collector with an `access_token` of kind `collector` (figure 11). Six trigram (GIN) indexes back global search; trigram matching was chosen over full-text search because PostgreSQL ships no Vietnamese or Chinese text-search configuration. Attribute lists in all four diagrams are abridged to the fields that matter architecturally.

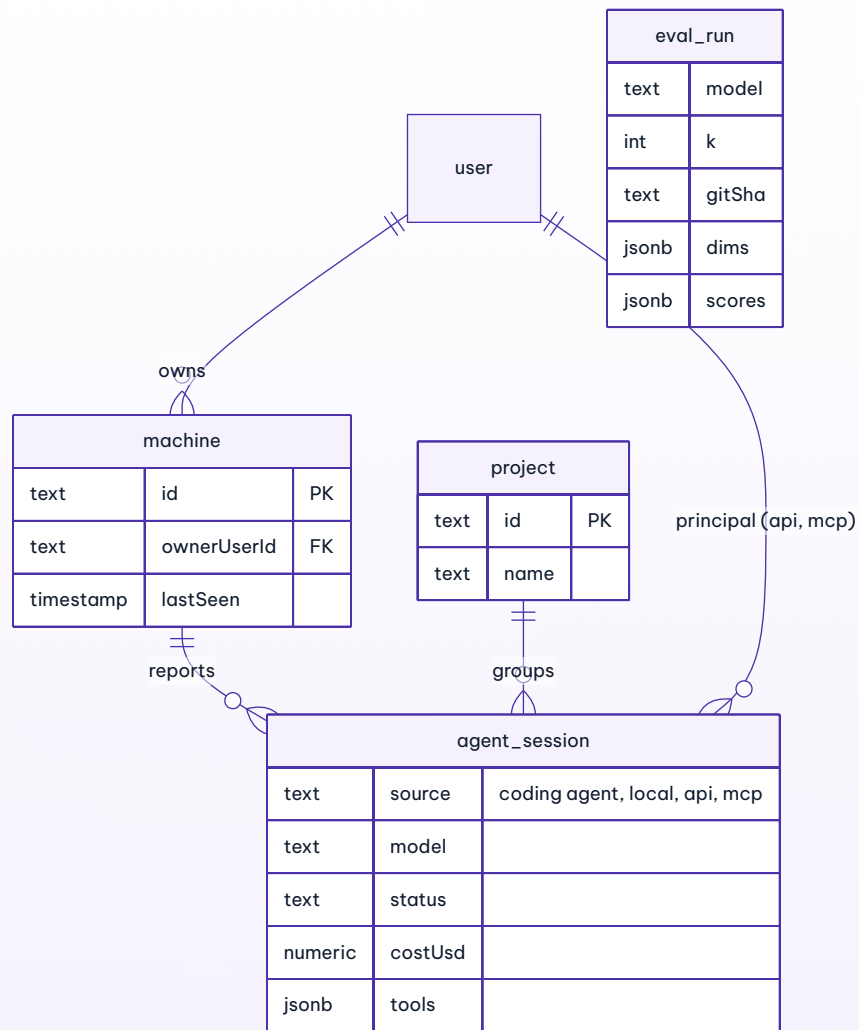


Figure 14. Monitoring and evaluation. `eval_run` is standalone: one row per offline evaluation run, charted on the Reliability page.

12 API surface

The HTTP API consists of 64 route files and 88 method handlers under `/api`. It serves the LAAM UI; three endpoints are designed for machine callers (collector ingest, the MCP server and the workflow tick). State-changing business routes require a mutating role (member or above); some personal and helper routes (AI drafting, document parsing, notifications, own profile) need only a session. Administrative routes require owner or admin.

Group	Routes	Scope	Auth
Auth and users	7	Auth.js handlers, registration (mode-gated, rate-limited), user administration, self-service profile, access tokens.	Public / session / owner-admin
Chat and assistant	21	Chat turn and confirm, model and tool catalogues, conversations, workspaces, custom agents, document parsing (OCR, PDF, DOCX), guarded URL fetch, TTS proxies, geo and weather proxies.	Session; mutations need member+
Connectors and MCP	5	Connector list, connect / disconnect / test / OAuth, per-user MCP servers and probe, LAAM's own MCP server.	Session; MCP server by token

Group	Routes	Scope	Auth
Monitoring	11	Collector ingest, local sync, SSE event stream, dashboard statistics, unified monitoring, session timeline, machines, host metrics, search, runtime config.	Session; ingest by token
Workflows	17	CRUD, run, clone, templates, runs and cancel, schedules, tick, AI generate / chat / review / append-write.	Session; tick by secret
Notifications	3	List, mark read, mark all read.	Session (own rows only)

■ Conventions

- **Ownership.** Per-user resources are filtered by the caller's id in the query itself, so another user's record behaves as not found rather than forbidden.
- **Validation.** Request bodies are validated with Zod or explicit parsers before any I/O; Claude model ids are checked against an allow-list.
- **Secrets never echoed.** Credential and MCP-server responses mask secrets to their last four characters; tokens are shown once at creation.
- **Streaming.** The chat turn streams framed text (chapter 4); live updates for sessions, workflow steps and notifications use SSE on `/api/events`, filtered per user.
- **MCP server.** `POST /api/mcp` implements `initialize`, `tools/list`, `tools/call` and `ping` as hand-written JSON-RPC over Streamable HTTP, exposing only read-only tools. No connector write is reachable through it.

The complete endpoint list is in the appendix.

13 Evaluation harness

Beyond unit tests, LAAM has an offline evaluation harness that runs the real system prompt and the real tool loop against real models, scores behaviour with automated graders, and records each run in the database for trend tracking.

■ Method

- **What is real.** The production `buildSystemPrompt` and `runToolRounds`, and a live model from the configured provider (BytePlus, Cerebras or Ollama; a missing key fails rather than falling back).
- **What is stubbed.** Tool dispatch. Tools return deterministic stub responses, so no connector or MCP server is called and runs are reproducible. Flag-off guards (drill-down) are not enabled.
- **Main suite.** 17 scenarios across tool selection, arguments, restraint (not calling tools for small talk), rich rendering, write intent, termination, web research and utilities. Each scenario runs k times; 8 graders score 7 dimensions. Each run is stored in `eval_run` with the git SHA and shown on the Reliability page.
- **Scale suite.** Measures tool-selection pass rate as the number of tools offered grows (4, 16, 32, 60), padding with a distractor pool drawn from the real tool union including a snapshot of a 48-tool MCP server, with Wilson 95% confidence intervals and no-call separated from wrong-call.

■ Latest measured results

Main suite, 2026-09-03: `gpt-oss-120b` on BytePlus, $k = 5$, 17 scenarios, 85 runs, stubbed dispatch.

Tool selection	Arguments	Grounding	Restraint	Termination	Write intent	Rich block
86%	97%	67%	100%	90%	100%	100%

Weak spots named by the scorecard: one scenario consistently prefers a digest tool over the expected statistics tool; one restraint scenario calls the wrong read tool; an open-ended web-research scenario runs 10–16 tool rounds before answering; one detail lookup terminates late. Grounding is the lowest dimension because the grader requires specific values from the stub to appear verbatim in the answer.

Scale suite, `gpt-oss-120b`: an early low-sample run ($k = 3$) suggested that tool selection stayed stable as the number of offered tools grew from 4 to 60; a later 60-tool run ($k = 8$, 2026-08-03) averaged 67%, so scaling is still being measured. At $k = 16$, the three highest-risk probes (voice-mode detail lookup, and two variants with a misleading earlier conversation) each passed 16 of 16, run without the drill-down safety net.

How to read these numbers

They are grader pass rates on scenarios with stubbed tool responses, not end-to-end accuracy on customer data. Real payloads are larger and noisier than stubs, and the harness does not exercise live connectors. The harness exists to catch regressions in prompt and loop behaviour when models, prompts or guards change, and to make those changes measurable.

Other test assets

363 test files cover the libraries, routes and UI components (Vitest with Testing Library). Notable invariant tests: the no-code-execution guard, durable workflow resume, tick authentication, UI contrast and i18n dictionary drift across the three languages. Additional A/B scripts compare providers and reasoning settings, and a replay harness re-runs multi-turn workflow-authoring conversations.

A Appendix – API endpoint reference

Auth legend: **Pub** public · **S** session · **M** session, mutations require member or above · **R** owner or admin · **T** bearer access token · **Tick** shared secret. All paths are under `/api`.

Method	Path	Purpose	Auth
Auth and users			
GET, POST	<code>/auth/[...nextauth]</code>	Auth.js handlers	Pub
POST	<code>/register</code>	Sign-up (open / invite / closed; 10 per hour per IP)	Pub
GET	<code>/users</code>	User list (safe column projection)	R
PATCH	<code>/users/[id]</code>	Change role (owner; last-owner guard) or disable (off-boarding transaction)	R
PATCH	<code>/users/me</code>	Edit own profile	S
GET, POST	<code>/access-tokens</code>	List own tokens; mint (admins may mint for another user)	M
DELETE	<code>/access-tokens/[id]</code>	Revoke	S
Chat and assistant			
POST	<code>/chat</code>	Streaming chat turn, or confirm / reject a pending write	M
GET	<code>/chat/info</code>	Default model and available model lists	S
GET	<code>/chat/tools</code>	Caller's tool catalogue (internal, connectors, MCP)	S
GET, POST	<code>/conversations</code>	List (search by title or content) / create	M
GET, PATCH, DELETE	<code>/conversations/[id]</code>	Messages with rebuilt tables / rename / delete	M
GET, POST	<code>/workspaces</code>	List / create chat workspaces	M
PATCH, DELETE	<code>/workspaces/[id]</code>	Edit / delete (conversations become ungrouped)	M

Method	Path	Purpose	Auth
GET, POST	/custom-agents	List / create presets	M
PATCH, DELETE	/custom-agents/[id]	Edit / delete	M
GET	/ollama/models	Local model list	S
POST	/fetch-url	Server-side URL fetch (hostname blocklist)	S
GET, POST	/ocr	OCR availability / recognise (vi, en, zh)	S
GET, POST	/pdf	PDF text extraction (text layer, then OCR, then vision)	S
GET, POST	/docx	DOCX text extraction	S
POST	/tts	Text-to-speech (WAV)	S
POST	/tts/stream	Streaming text-to-speech (PCM)	S
GET	/geocode , /reverse , /route , /nearby	Geocoding, reverse geocoding, routing, nearby places (cached, throttled)	S
GET	/weather	Current weather	S
Connectors and MCP			
GET	/connectors	Connector list with masked credentials	S
GET, POST	/connectors/[id]/[action]	Connect / disconnect / test / OAuth start and callback	M
GET, POST, PATCH, DELETE	/connectors/mcp	Per-user MCP servers (token never echoed)	M
POST	/connectors/mcp/test	Probe a server before saving	S
POST	/mcp	LAAM as MCP server (read-only tools)	T
Monitoring			
POST	/ingest	Collector push (5 MB streamed body cap)	T
POST	/sync	Scan local transcripts into the database	M
GET	/events	SSE: sessions, workflow steps, per-user notifications	S
GET	/stats	Dashboard aggregates	S
GET	/monitoring	Unified monitored runs (agent, chat, workflow)	S
GET	/agents/[id]/timeline	Session message and tool log	S
GET, POST	/machines	List / create machine and issue collector token	S / R
DELETE	/machines/[id]	Revoke collector token	R
GET	/host/metrics	Host CPU, RAM, GPU (fail-soft)	S
GET	/config	Client runtime configuration	S
GET	/search	Global search	S
Workflows			
GET, POST	/workflows	List (paged, with total) / create	M
GET, PATCH, DELETE	/workflows/[id]	Read / update / delete (cascades schedules and runs)	M
POST	/workflows/[id]/run	Run now; dry-run; draft graph (dry-run only)	M
POST	/workflows/[id]/clone	Clone own workflow or template	M
POST	/workflows/[id]/save-as-template	Save graph as template	M
GET, POST	/workflows/templates	Template catalogue / instantiate	M
POST	/workflows/templates/[id]/instantiate	Instantiate a template	M
GET	/workflows/runs	Caller's runs (filter by workflow, status, latest)	S
GET, PATCH	/workflows/runs/[id]	Run detail with steps / cancel	M
GET, POST	/workflows/schedules	List / create schedule (cron validated)	M
PATCH, DELETE	/workflows/schedules/[id]	Enable, modify, delete	M
POST	/workflows/tick	Claim due schedules, execute queued and resumable runs	Tick
POST	/workflows/generate	One-shot AI generation (probe, plan, verify)	S
POST	/workflows/generate/chat	Multi-turn AI authoring turn	S
POST	/workflows/generate/compile	Compile an observed trace into a graph (legacy path)	S
POST	/workflows/generate/append-write	Append a human-confirmed write step	S
POST	/workflows/review	Plain-language review of the current graph	S
Notifications			
GET	/notifications	Caller's notifications and unread count	S
PATCH	/notifications/[id]	Mark one read	S
POST	/notifications/read-all	Mark all read	S